Vector Databases, Explained

What they are, how similarity search works, and when you actually need one

---

A regular database answers a precise question: does a record matching these exact criteria exist? A vector database answers a different, fuzzier question: which records are most similar in meaning to this one? That shift, from matching exact text to matching meaning, is the quiet engine behind semantic search, retrieval-augmented generation (RAG), recommendation systems, and most forms of AI "memory." This article covers how the underlying ideas work, the trade-offs between the common approaches, and a grounded view of when a dedicated vector database actually earns its keep.

What an embedding is

An embedding is a list of floating-point numbers, typically a few hundred to a few thousand values long, that represents a piece of content — text, an image, audio, code — in a way that captures its meaning rather than its exact form. An embedding model is trained so that content with similar meaning ends up with vectors that sit close together in that numeric space, even when the surface wording is completely different. The classic example: "how to fix a flat tire" and "changing a punctured wheel" share almost no words, but a good embedding model places their vectors near each other, because semantic similarity becomes geometric distance.

Popular embedding models in 2026 include OpenAI's text-embedding-3 family, Cohere's embed-v4, Anthropic's Voyage embeddings, and a range of open-source options such as Nomic, BGE, and E5. Most production systems pick one model and standardize on it, since switching embedding models later means re-embedding the entire dataset.

What a vector database actually does

At its core, a vector database is a specialized store built around one operation: nearest-neighbor search. Given a query vector, return the K stored vectors closest to it. That single operation is what powers semantic search, "find similar documents," and every retrieval step in a RAG pipeline.

The naive way to do this — compute the distance from the query to every stored vector — is computationally linear in the size of the dataset, and it stops being practical well before a million vectors. Vector databases instead build an index that supports approximate nearest-neighbor (ANN) search: they trade a small, tunable amount of accuracy for a large improvement in query speed.

The two dominant indexing approaches

HNSW (Hierarchical Navigable Small World). Builds a multi-layer graph in which each vector is connected to its nearest neighbors. A query starts at the coarse top layer and descends through

progressively finer layers, narrowing the search space at each step. HNSW typically delivers high recall — often in the 95–99% range — but its memory footprint is higher than alternatives, which is why most production systems reach for it at scales under roughly 100 million vectors.

IVF (Inverted File Index), often paired with Product Quantization (PQ). Partitions the vector space into clusters using k-means; at query time, only the nearest few clusters are searched instead of the whole dataset, typically covering 5–10% of the data for roughly 90–95% recall. Product Quantization compresses each vector by splitting it into sub-vectors and replacing each with the index of its nearest codebook entry, which can cut memory use by 10–50x. IVF-PQ is the more common choice once a collection grows past the point where HNSW's memory cost becomes unwieldy — roughly the hundred-million-vector-plus range.

A third approach, locality-sensitive hashing (LSH), groups vectors into hash buckets so that similar vectors are more likely to land in the same bucket. It's less commonly the default choice in 2026 than HNSW or IVF, but still shows up in some systems, particularly for very high-dimensional or streaming use cases.

Do you actually need a dedicated vector database?

This is the question most teams skip. A meaningful share of projects that reach for a dedicated vector database don't need one yet. A few honest rules of thumb:

- **Small collections (under roughly 50,000 vectors):** brute-force distance computation is often fast enough that you don't need ANN indexing at all.
- **Already running Postgres:** the pgvector extension adds vector similarity search directly to a database you likely already operate. You get full SQL, ACID transactions, and the ability to join vector search results against your existing relational data — at the cost of somewhat less specialized performance at very large scale.
- **Large scale, dedicated workload:** once a collection reaches into the millions of vectors with tight latency requirements, or the access pattern is vector-search-first rather than an add-on to relational queries, a purpose-built vector database starts to pay for itself.

A quick look at the field (mid-2026, vendor-neutral)

System	Notes
Pinecone	Fully managed; built to search billions of items with low latency.
Qdrant	Written in Rust; applies metadata filtering during graph traversal rather than as a separate pre/post
Milvus	Kubernetes-native, built for billion-scale collections with GPU acceleration.
Chroma	Apache 2.0, Rust-based; the easiest option to start with, runs embedded or client-server.

pgvector	A Postgres extension rather than a standalone system; the "boring technology" choice for teams al

These systems differ less in raw capability than in operating model, ecosystem fit, and how much infrastructure you want to manage yourself. The right choice depends on scale, existing stack, and operational budget — not on whichever vendor has the loudest benchmark post.

A note on architecture

Vector databases are generally best treated as an index, not a system of record. A common and sound pattern is to store the embedding, a chunk identifier, and minimal metadata in the vector store, while keeping the full document content, user data, and application state in a primary database. That keeps the vector store focused on what it's good at — fast similarity search — and avoids treating it as a general-purpose data store it wasn't designed to be.

Compiled from public technical documentation and industry guides current as of mid-2026, including IBM, DigitalOcean, and multiple vendor-neutral practitioner guides. Figures on recall, memory savings, and scale thresholds are representative ranges reported across sources, not benchmarks run for this document.